Autonomous Mapping, Navigation & Control System for a UGV

Technical explanation · November 2026

ROS 2 · LiDAR · SLAM · AUTONOMOUS CONTROL

01 Overview and Assumptions

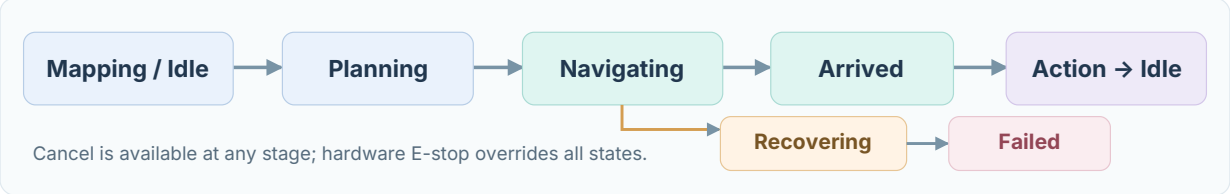
> The system lets an operator watch a live LiDAR-built map and camera feed on the UGV's display, tap a destination on the map (for example, a spot in another room), and have the vehicle plan a safe route, drive there autonomously while avoiding obstacles, confirm arrival, and carry out a task.

The design assumes a typical small differential-drive UGV:

COMPONENT	EXAMPLE	ROLE
2D 360° LiDAR	RPLIDAR A2/S2, ~10 Hz, 0.15–12 m	Mapping, localization, obstacle detection
Camera	USB/CSI, 640×480 @ 30 fps	Live view, optional object detection at goal
Wheel encoders + IMU	Quadrature encoders, 6-axis IMU	Odometry and heading
Motor controller	STM32/ESP32 + dual H-bridge (or RoboClaw)	Closed-loop wheel speed control
Onboard computer	Jetson Orin Nano / Raspberry Pi 5	Runs perception, planning, GUI
Display	7–10" touchscreen	Operator interface

The software is built on **ROS 2** (Humble/Jazzy), which provides the message bus, coordinate-frame management (TF2) and mature packages for SLAM and navigation, with custom nodes for the GUI, goal handling and motor interface.

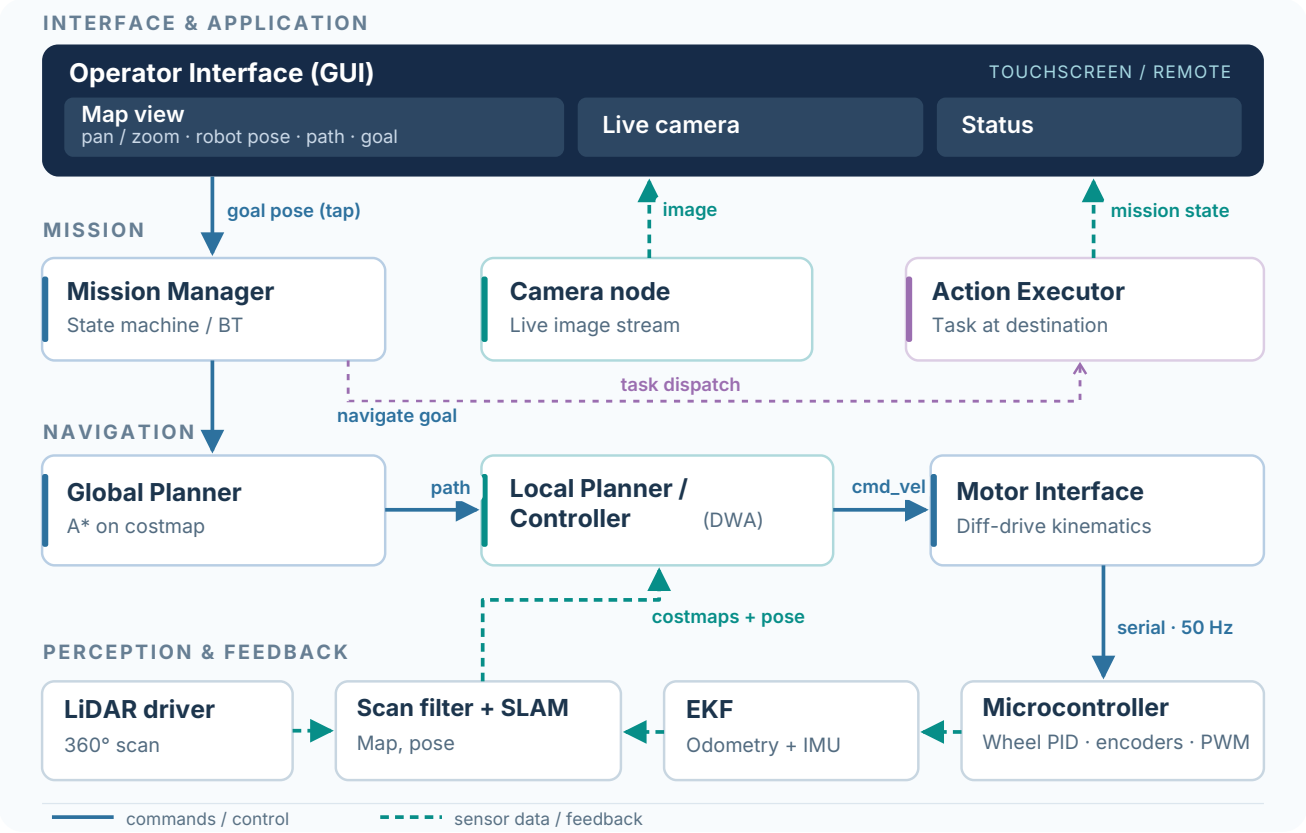
AT A GLANCE / MISSION PROGRESSION



02 Overall Software Architecture and Workflow

The software is organised in layers, each a set of independent ROS 2 nodes communicating through topics and actions. This keeps the high-rate, safety-critical motor loop on a microcontroller and the heavier perception and planning work on the main computer.

FIGURE 01 / SYSTEM ARCHITECTURE



ROS 2 node layers and the primary information, command and feedback paths.

NOMINAL UPDATE RATES

LiDAR **10 Hz**	SLAM map **1–2 Hz**	EKF **50 Hz**	Local planner **10–20 Hz**
Motor PID on MCU **500 Hz–1 kHz**	Camera **30 fps**	GUI refresh **20–30 fps**	

Workflow. On power-up the drivers start and the robot enters *Mapping/Idle*: SLAM builds the map as the robot moves (manually driven or exploring), and the GUI shows the map and camera side by side. When the operator taps a destination, the Mission Manager validates it and moves to *Planning*; the global planner computes a route, and the system enters *Navigating*, where the local controller follows the path and reacts to obstacles. If progress stalls it enters *Recovering*; if recovery fails it reports *Failed* and asks the operator for help. When the goal tolerance is met it enters *Arrived*, the Action Executor runs the assigned task, and the robot returns to *Idle* (or optionally drives home). The operator can cancel at any time, and a hardware E-stop overrides everything.

03 LiDAR Data Processing and Real-Time Map Generation

Pre-processing. Each scan arrives as a list of ranges at fixed angular steps. The scan filter discards readings outside the sensor's valid range, masks out angles that hit the robot's own body, removes isolated outliers with a small median filter, and compensates motion distortion by using odometry to shift each beam to where the robot was when that beam was taken (important when turning, since one revolution takes 100 ms). Each beam is then converted from polar to Cartesian coordinates in the robot frame, `x = r·cos(θ)`, `y = r·sin(θ)`, and transformed into the map frame using the current pose estimate.

SLAM. Mapping and localization are solved together with graph-based SLAM (e.g. `slam_toolbox`). Each new scan is aligned to the existing map by correlative scan matching, which corrects odometry drift. Scans are stored as nodes in a pose graph; when the robot recognises a previously visited area, a loop-closure constraint is added and the graph is optimised, straightening the whole map.

Occupancy grid. The map is a 2D grid (5 cm cells) where each cell stores the log-odds that it is occupied. For every beam, cells along the ray from the sensor to the hit point (traced with Bresenham's line algorithm) are marked more likely free, and the end cell more likely occupied:

PYTHON OCCUPANCY-GRID LOG-ODDS UPDATE

```python
L_FREE, L_OCC, L_MIN, L_MAX = -0.4, 0.85, -4.0, 4.0
for cell in bresenham(sensor_cell, hit_cell)[:-1]:
    grid[cell] = max(L_MIN, grid[cell] + L_FREE)
if beam_hit_obstacle:  # not a max-range reading
    grid[hit_cell] = min(L_MAX, grid[hit_cell] + L_OCC)
# probability for display: p = 1 - 1 / (1 + exp(L))
```

The log-odds form makes updates a simple addition and naturally filters noise and moving objects over repeated scans. For display, cells are rendered as white (free), black (occupied) and grey (unknown), and the map image is redrawn whenever SLAM publishes an update. A saved map can later be reloaded so the robot only needs to localize, not re-map.

Map display legend Free Occupied Unknown

04 Camera Integration and Live Visualization

The camera node captures frames through V4L2/GStreamer, uses the hardware encoder to compress them (JPEG or H.264), and publishes them as compressed images, which keeps CPU and bandwidth low and allows the same stream to be viewed remotely. The GUI decodes frames on a background thread so video never blocks map interaction, and drops stale frames rather than queueing them, holding glass-to-glass latency under roughly 150 ms.

The GUI (built with Qt/PySide6 for the onboard screen, or as a web page over `rosbridge` for remote use) shows the map on one side and the camera on the other, with a status bar for mission state, battery, speed and localization quality. The map panel overlays the robot's pose and heading, the live LiDAR points, the

planned global path, the local trajectory and the selected goal. The camera panel can overlay the heading, distance to goal and, with a calibrated camera–LiDAR extrinsic transform, LiDAR points projected onto the image so the operator can see what the robot treats as an obstacle.

05 Map-Based Destination Selection

When the operator taps the map, the GUI first undoes its own pan/zoom to get the grid cell `(u, v)`, then converts that cell to world coordinates using the map's resolution and origin (image rows run downward, map *y* runs upward):

PYTHON MAP PIXEL TO WORLD COORDINATES

```python
x = map.origin_x + (u + 0.5) * map.resolution
y = map.origin_y + (map.height - 1 - v + 0.5) * map.resolution
```

Dragging from the tapped point sets the desired final heading; a plain tap leaves heading free. Before the goal is accepted, it is checked against the inflated costmap. If it falls on an obstacle or too close to a wall, the system searches outward (breadth-first) for the nearest safe cell and proposes it, highlighting the adjusted point. If it lies in unknown space, such as a room not yet scanned, the operator can choose an *explore-to* mode in which the planner treats unknown cells as passable at extra cost and the map fills in as the robot approaches. The accepted goal is sent to the navigation stack as a `NavigateToPose` action, which streams progress (distance remaining, ETA) back to the GUI and can be cancelled.

06 Localization, Path Planning and Obstacle Avoidance

Localization. Wheel odometry and IMU yaw rate are fused in an Extended Kalman Filter (`robot_localization`) to give a smooth, high-rate pose that drifts slowly. SLAM corrects that drift at scan rate by publishing the map→odom correction. In a previously saved map, AMCL (a particle filter matching scans to the map) can be used instead. Frames follow the standard tree `map → odom → base_link → laser / camera`, so every component can express data in whatever frame it needs.

Costmaps. Planning uses layered costmaps: a static layer from the SLAM map, an obstacle layer updated from live scans, and an inflation layer that spreads a decaying cost around obstacles out to the robot radius plus a safety margin. Cells closer than the robot radius are lethal; nearby cells are expensive, so routes naturally keep to the middle of corridors and doorways.

Global planning. An A* search on the 8-connected grid finds the route from the robot to the goal, with step cost = distance + *w* × cell cost and an octile-distance heuristic, which yields the shortest *safe* path rather than the shortest geometric one. The raw path is smoothed to remove grid zig-zags. Replanning happens about once per second or immediately when the current path becomes blocked. (Hybrid A* or Smac planners can be swapped in for robots with turning constraints.)

Local planning and avoidance. The local controller, here the Dynamic Window Approach (DWA) running at 10–20 Hz on a 4 m × 4 m rolling costmap built from the latest scans, samples feasible linear and angular velocity pairs within the robot's acceleration limits, simulates each for about 1.5 s, discards any that collide, and scores the rest by alignment with the global path, progress toward the goal, clearance from obstacles

and speed. The best pair is sent as a velocity command. Because the local costmap uses raw scans, it reacts to people, doors and objects that were not in the map. (Regulated Pure Pursuit or MPPI are good alternatives.)

Recovery. If the robot makes no progress for several seconds, it escalates through recovery behaviours: clear the local costmap, rotate in place to re-sense, back up slightly, and replan. If those fail, it stops and notifies the operator.

07 Motor Control, Autonomous Navigation and Destination Confirmation

From velocity commands to wheels. The planner outputs a linear velocity v and angular velocity ω. The motor interface converts these with differential-drive kinematics, where L is the wheel separation and r the wheel radius:

PYTHON DIFFERENTIAL-DRIVE KINEMATICS

```python
v_left  = v - omega * L / 2
v_right = v + omega * L / 2
w_left, w_right = v_left / r, v_right / r     # wheel speeds, rad/s
```

This single mapping covers every manoeuvre: equal positive speeds drive forward, equal negative speeds reverse, opposite speeds turn on the spot, and unequal speeds produce curves and fine heading corrections. Speeds are clamped to limits and ramped to respect acceleration limits, then sent over serial to the microcontroller at about 50 Hz.

Low-level control. The MCU runs a PID speed loop per wheel at 500 Hz–1 kHz, comparing the target speed with encoder feedback and adjusting the PWM duty cycle and direction pins on the H-bridge, so the wheels hold the commanded speed regardless of load or slope. The MCU also returns encoder counts for odometry. Several safety layers sit around this: a watchdog stops the motors if no command arrives for 200 ms, a collision monitor halts or slows the robot if any LiDAR point enters a stop or slow-down zone around the body regardless of what the planner says, and a hardware E-stop cuts motor power directly.

Navigation loop. Putting it together, the navigation cycle runs continuously while a goal is active:

PYTHON NAVIGATION CONTROL LOOP

```python
while mission.active:
    pose = tf.lookup("map", "base_link")
    if goal_reached(pose, goal): break
    if path_blocked() or time_to_replan(): path = global_planner.plan(pose, goal)
    v, omega = local_planner.compute(pose, path, local_costmap)
    if not making_progress(): run_recovery()
    motor_interface.send(v, omega)
motor_interface.send(0, 0)
```

Destination confirmation. The goal counts as reached when the robot is within 0.15 m of the target position and, if a heading was set, within about 6° of it, held for 0.5 s with near-zero velocity to avoid false triggers. The Mission Manager then switches to *Arrived*, the GUI shows a confirmation banner with the final position error and time taken, a sound plays, and a camera snapshot is saved and logged with the pose.

Performing the action. The Action Executor runs a plugin selected with the goal. Examples include capturing a photo or a 360° panorama by rotating in place, running an object detector on the camera feed to verify or locate a target (and fine-aligning to it using visual servoing if needed), operating a payload such as a servo-driven gripper or dispenser, or waiting for operator confirmation. When the action completes, the result is reported to the GUI and the robot returns to *Idle*, ready for the next destination or a "return home" command.

08 Testing, Safety and Extensions

Development starts in simulation (Gazebo with a model of the robot and a multi-room world) to tune SLAM, costmap and DWA parameters safely, then moves to recorded data (rosbag replay of real scans) and finally to staged field tests at low speed limits. Unit tests cover the coordinate transforms, A* on known grids and the kinematics. Natural extensions include autonomous frontier-based exploration to map new areas without manual driving, multi-goal patrol routes, 3D LiDAR or depth cameras for overhanging obstacles, and semantic labels on the map (for example "kitchen") so destinations can be chosen by name.

DESIGN QUICK REFERENCE

Condensed from Sections 1–8

Perception & mapping

2D LiDAR	**360° · ~10 Hz**
Camera stream	**640×480 · 30 fps**
Occupancy cell	**5 cm**
SLAM map refresh	**1–2 Hz**

Navigation & localization

EKF	**50 Hz**
Local planner	**10–20 Hz**
Rolling costmap	**4 m × 4 m**
DWA simulation	**~1.5 s**

Drive & protection

Serial commands	**~50 Hz**
Wheel PID	**500 Hz–1 kHz**
Command watchdog	**200 ms**
Manual override	**Hardware E-stop**

Arrival & test validation

Position tolerance	**0.15 m**
Heading tolerance	**~6°**
Arrival hold time	**0.5 s**
Field validation	**Staged, low speed**

VALIDATION PATH Gazebo simulation → rosbag replay → Staged field tests

These figures are the illustrative design parameters stated in the source document, not independently validated performance guarantees.